

Aula 17 - Introdução a Árvores

Árvores

Dentro da Teoria dos Grafos, Árvores são definidas como grafos que:

1. São conexos
2. São acíclicos (não possuem nenhum ciclo)

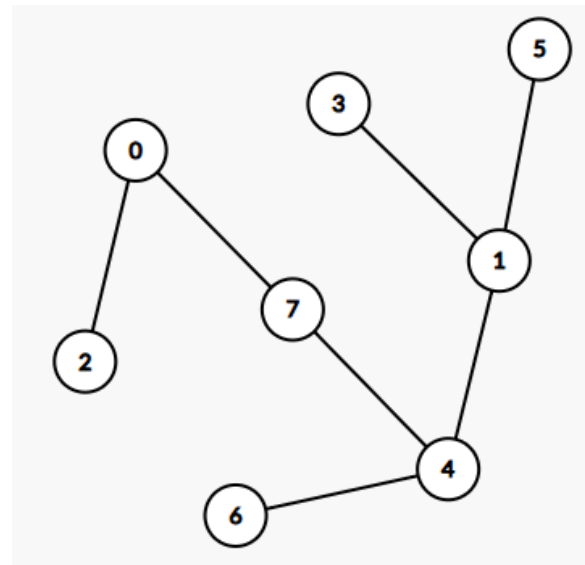
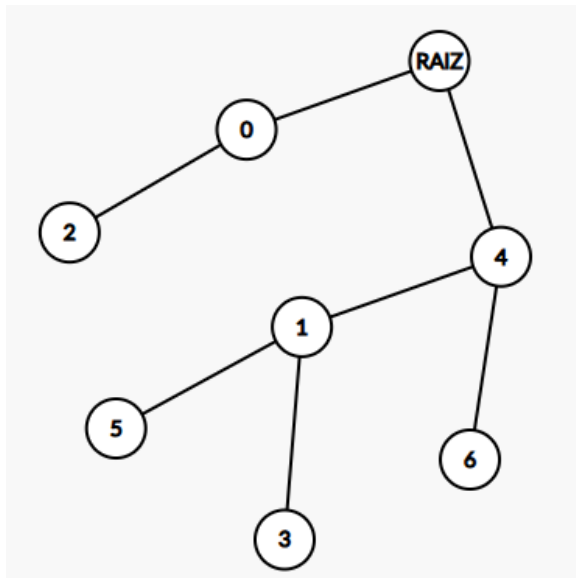
Partindo de um grafo com essas duas características, conseguimos provar, também, algumas outras características muito úteis.

Na prática, podemos dizer que uma árvore é um grafo que:

1. É conexo
2. É acíclico
3. Se tem N vértices, ele tem exatamente $N - 1$ arestas
4. Existe **exatamente** um caminho entre quaisquer par de vértices u e v
5. Remover qualquer aresta dentro desse grafo torna ele desconexo

Um fato interessante é que, partindo de quaisquer duas dessas cinco características das árvores, conseguimos chegar em todas as outras. Então, se um grafo possuir duas características dessas, assuma que todas as outras também são verdade, e usem delas para possivelmente resolver o problema. (Por exemplo, sabendo que uma árvore não tem mais de um caminho entre dois vértices, saber a distância de um vértice para todos os outros pode ser resolvido só com uma BFS ao invés de um Dijkstra).

Via de regra, conseguimos pensar em árvores sendo enraizadas ou não enraizadas:



Ambos os grafos acima são exatamente iguais, mas visualizados de forma diferente. A existência ou não de uma “raiz” para a árvore (um vértice de partida) é puramente conceitual (o problema define).

Na programação competitiva, podemos abordar muitos problemas envolvendo árvores. Essa aula aborda alguns deles (os mais básicos, e mais comuns).

DFS/BFS comum

Aproveitando para relembrar como funcionam os algoritmos básicos de busca, em árvores, eles normalmente aparecem levemente modificados.

Na forma recursiva, como não há mais de uma forma de sair de um vértice v para um vértice u , o vetor de visitados perde utilidade (se chegamos em u na busca, já temos certeza que não chegaremos nele de novo, então não faz sentido checar se ele já foi visitado).

Todavia, a maioria dos problemas de árvores modela-as como grafos não direcionados. Isso introduz um perigo: visitar um nó pai sem querer. Na prática, isso significaria “começar na raiz, e ir descendo, mas em determinado momento, subir na árvore sem querer”.

Dessa forma, a única verificação que precisamos fazer é: ao olhar para os vizinhos u de um determinado vértice v , temos que checar se u não é pai de v antes de prosseguir.

Em uma DFS recursiva, podemos fazer isso passando o “pai” de v como parâmetro para a função:

```
void dfs(int v, int pai)
    for (int u : adj[v]) {
        if (u == pai) continue;

        dfs(u, v); // passamos v como "pai" de u
        // para garantir que não vamos "subir" na árvore
    }
}
```

Aí, surge a pergunta, e na chamada inicial? Quem é o pai da raiz? É só colocar um valor que você tem certeza que não representa nenhum vértice do grafo, tipo

```
dfs(raiz, -1).
```

Já na DFS/BFS iterativa, teríamos que saber quem é o pai de cada um dos vértices. Embora isso seja possível, é mais simples só voltar para o uso do vetor de visitados.

```
queue<int> q;
q.push(root);

visited[root] = true;

while (!q.empty()) {
    int v = q.front();
    q.pop();

    for (int u : adj[v]) {
        if (visited[u]) continue;

        visited[u] = true;

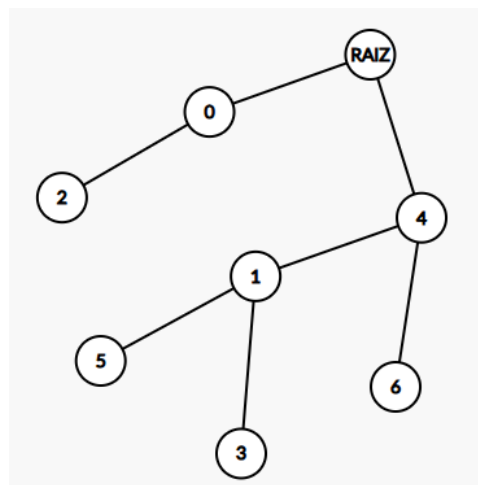
        q.push(u);
    }
}
```

```
}  
}
```

Profundidade de um vértice

Um problema muito comum é saber qual a “profundidade” de um vértice. Isso é basicamente a distância entre a raiz e aquele determinado vértice.

Em árvores enraizadas, podemos enxergá-las com **níveis** de profundidade:



Nesse grafo, por exemplo, o vértice 2 está no mesmo nível que o vértice 1 e o vértice 6. Enquanto os vértices 3 e 5 também estão no mesmo nível.

Esse problema é resolvido tranquilamente com uma busca a partir da raiz:

```
vector<int> profundidades(n);  
profundidade[raiz] = 0;  
  
void dfs(int v, int pai){  
    for (int u : adj[v]) {  
        if (u == pai) continue;  
  
        profundidade[u] = profundidade[pai] + 1; // linha nova!  
    }
```

```

        dfs(u, v);
    }
}

```

Notem que, ao colocar o `profundidade[u] = profundidade[pai] + 1;` dentro do `for` que analisa os vizinhos do vértice `v`, a profundidade da raiz não será atualizada. Por isso, precisamos atualizá-la de antemão, e também não teremos o problema de acessar `profundidade[-1]` ao chamar `dfs(raiz, -1)`.

A versão iterativa desse problema também é trivial, basta incluir a mesma linha que incluímos na versão recursiva, no lugar certo. Eu recomendo vocês tentarem fazer sozinhos antes de ver a resposta certa:

```

queue<int> q;
q.push(root);

visited[root] = true;
profundidade[root] = 0;

while (!q.empty()) {
    int v = q.front();
    q.pop();

    for (int u : adj[v]) {
        if (visited[u]) continue;

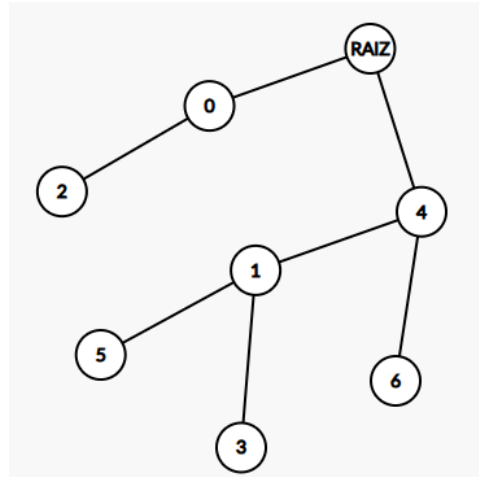
        visited[u] = true;
        profundidade[u] = profundidade[v] + 1;

        q.push(u);
    }
}

```

Tamanho da subárvore de um vértice

Quando temos árvores enraizadas, podemos pensar, também, em **subárvores**. Uma subárvore de um vértice v , é, basicamente, todos os filhos (e netos, bisnetos...) de v , mais o próprio v .



A subárvore do vértice 4 tem 5 vértices. Já a subárvore do vértice 1 tem 3 vértices. E a subárvore do 5 tem só um vértice.

Em problemas de programação competitiva, isso aparece contextualizado como “todos os subordinados de um funcionário” ou “todos os descendentes de uma pessoa”. Obviamente, nesses dois casos, fica o tamanho da subárvore -1 .

Esse problema também é resolvido incluindo uma linha nova em uma busca comum. Todavia, aqui a gente nota um padrão muito interessante. No problema de contar a profundidade de um vértice, nós começávamos contando da raiz (0) e íamos descendo contando $+1, +1, +1...$ Agora, o certo vai ser começar das **folhas** (os vértices sem nenhum filho) e ir subindo contando $+1, +1, +1...$

Na DFS recursiva, isso implica só em mudar **onde** a linha nova entra:

```
vector<int> subarvore(n);

void dfs(int v, int pai){
    subarvore[v] = 1; // precisamos setar o valor inicial
    ao entrar no vértice
    // que no começo é só o próprio vértice
```

```

for (int u : adj[v]) {
    if (u == pai) continue;

    dfs(u, v);

    subarvore[v] += subarvore[u]; // linha nova!
}
}

```

A diferença entre essa DFS e a DFS do problema das profundidades é extremamente difícil de entender e de enxergar, recomendo que vocês invistam bastante tempo só analisando e digerindo esses dois algoritmos e a diferença entre eles.

Quando chamamos a atualização do vetor **antes** da chamada recursiva `dfs(u, v)`, nós atualizamos a informação que temos sobre o vértice atual (v) e passamos adiante para cada vizinho u . A atualização da informação sobre os vizinhos u , em contrapartida, vai depender da informação que temos do seu pai (o vértice v). Logo, a ordem que fizemos (de atualizar primeiro o pai e depois cada filho) é correta no problema das profundidades.

Já no problema das subárvores, a informação que temos de um vértice v depende da informação que temos dos seus filhos u , e dos filhos de seus filhos, e dos filhos de seus netos... Assim, precisamos primeiro realizar a chamada recursiva (para mergulhar o máximo possível no grafo) para aí sim, ir atualizando todo mundo na ordem contrária à que fizemos anteriormente.

A versão iterativa aqui fica extremamente contraintuitiva (porque simular esse comportamento de percorrer o grafo "ao contrário" não é tão simples quanto mudar a posição de uma linha). Se vocês tiverem curiosidade para saber como seria isso, pesquisem por fora, mas não recomendo.

Conclusão dos dois últimos problemas

Esses dois problemas representam toda uma classe de problemas que pode ser resolvida com uma modificação em uma DFS/BFS padrão (principalmente a DFS,

e especificamente a versão recursiva), tomando sempre os devidos cuidados no posicionamento da linha de atualização.

Diâmetro de uma árvore

O diâmetro de um grafo é definido como a maior distância possível entre dois vértices quaisquer nesse grafo. A solução para esse problema em grafos gerais consiste no uso do algoritmo de Floyd-Warshall (um que não vimos ainda). Porém, em árvores, a solução fica um pouco mais fácil.

Achar um diâmetro em uma árvore pode ser tão simples quanto: primeiro, rodar uma DFS em qualquer vértice da árvore para descobrir o vértice mais distante possível, em seguida, rodar outra DFS partindo desse vértice para achar a outra “ponta” mais distante da árvore. O diâmetro será a distância entre esses dois vértices descobertos.

Para tal, precisamos modificar a DFS para que ela descubra quem é o vértice mais afastado possível:

```
void dfs_maisLonge(int v, int pai, int dist, int &verticeMais
Longe, int &maiorDist) {
    if (dist > maiorDist) {
        maiorDist = dist;
        verticeMaisLonge = v;
    }

    for (int u : adj[v]) {
        if (u == pai) continue;
        dfs_farthest(u, v, dist + 1, verticeMaisLonge, maiorD
ist);
    }
}
```

Depois, encapsulamos essa função e chamamos ela duas vezes:

```
int diametro() {
    int A = 1;
```



```
int distA = -1;

dfs_maisLonge(1, -1, 0, A, distA);

int B = A;
int ddiametro = -1;

dfs_maisLonge(A, -1, 0, B, diametro);

return diametro;
}
```

Daí, descobrimos os dois vértices A e B que representam as “pontas” mais afastadas da árvore, e a distância entre eles é o próprio diâmetro.